

Senior QA Automation Engineer

Practical Test Task · Limestone Digital

This task takes one hour. We are not looking for coverage, we are looking at how you structure a suite and why. A small, well-designed solution with a clear account of the architecture around it scores higher than a large one.

Part 1 — Automation framework

Build a framework from scratch with a couple of UI tests and API tests. The targets are fixed so that we can compare submissions fairly:

Layer	System under test	Suggested scenario
UI	https://www.saucedemo.com/	Log in, add an item to the cart, verify the cart
API	https://jsonplaceholder.typicode.com/	GET a resource, assert the response contract

C# / .NET, with NUnit or xUnit. Selenium WebDriver for the UI test and RestSharp for the API test. SpecFlow is welcome but not required. The tests do not have to be green on our machine — the structure and the reasoning matter more to us than a passing run.

README

- How to install and run: the full suite, and a single test.
- How to open the report or where the test results land.
- A short **Design decisions** section — five lines is enough: which patterns you used and why, what you deliberately skipped, and what you would add with more time.

Framework structure

An hour is not enough to build everything a long-lived suite needs, so describe the rest rather than building it. In the README, set out the framework you would put in place for a suite that has to survive years of a real project, and how the pieces fit together:

- The layers you would have and what each one owns.
- Where driver lifecycle and browser configuration live, and how you would run the same suite locally, in a container, and on a grid.
- Where environment configuration and secrets live, and how the suite is pointed at a different environment.
- Where page objects, step definitions, test data and API clients belong, and what is not allowed to leak between them.
- How you keep tests independent and runnable in parallel.
- How failures are reported and investigated, and how you tell a test problem from a product bug.
- How the suite is integrated into CI and what it is allowed to block.

Say plainly which parts exist in your submission and which are description only, and name the improvements you would make first, in order. **This is the section we read most closely.**

Part 2 — Postman collection

Create a collection with these two GET requests:

- GET <https://jsonplaceholder.typicode.com/users> — the list of users.

- GET <https://jsonplaceholder.typicode.com/posts?userId=1> — the posts of one user.

Tests required for both requests:

- Status code is 200.
- Loop through the returned array and assert that the key properties of every object are not empty, null or undefined.

Use a Postman environment with a `baseUrl` variable instead of hard-coding the host. Export the collection and the environment and attach both.

Part 3 — SQL

In the W3Schools playground — Try-It editor — find all customer names and countries that used United Package as their shipper. Send us the query you used.

Submission

- A link to a public Git repository (bin, obj and packages excluded).
- Attach the exported Postman collection and environment, and the SQL query.
- Send to anna.sidnenko@limestonedigital.com and mariana.andruk@limestonedigital.com in the same email.
- **Subject: QA Test Task — <Your Name>**

At a glance

Time	1 hour. If you do not finish, send what you have.
Deliverable	A link to a public Git repository.
Stack	C# / .NET with NUnit or xUnit, Selenium WebDriver and RestSharp. SpecFlow optional.
AI	AI assistants are allowed. You will be asked to explain your choices on a short follow-up call.

Optional — only if time remains

One of these is plenty: a SpecFlow feature file for the UI scenario · a test data builder or factory · response-schema validation · a smoke category or trait · a Dockerfile for the test environment · a WireMock stub for one dependency · a CI workflow.

Skipping all of them costs you nothing if your Design decisions and Framework structure sections explain why.

If the hour runs out

Send the task as it is. A README line saying what is unfinished and how you would finish it counts in your favour.

If anything here is ambiguous, make a reasonable assumption and write it down — documented assumptions are part of what we are testing for.